

Kampanya Öneri Motoru — Teknik Rapor ve Doğrulama

Proje: CampaignSystem (banch_kampanya_sistemi) · **Dal:** feature/campaign-recommendations · **PR:** #54
Tarih: 2026-09-01

Bu belge iki şeyi kapsar:

- Bölüm A — Motor nasıl çalışır:** GET /api/campaign-recommendations uç noktasının arkasındaki mantık, koddan adım adım.
- Bölüm B — Doğrulama:** sıfırdan üretilmiş ~50.000 satırlık gerçekçi bir harcama veri setiyle motorun test edilmesi; aynı veriden motora bakmadan, farklı istatistiksel yöntemlerle bağımsız bir sıralama çıkarılıp ikisinin karşılaştırılması.

Yönetici Özeti

Ne yapar. Operatöre, son dönem kart harcamalarına bakarak hangi merchant kategorisinde kampanya açmaya değer olduğunu sıralı bir liste hâlinde sunar. Liste hesaplanır, saklanmaz — her istekte yeniden üretilir. Bir öneriye tıklanınca kampanya formu önerilen değerlerle dolar. Öneriler artık hem ayrı "Kampanya Önerileri" ekranında hem de **kampanya oluşturma formunun içinde** otomatik görünür.

Nasıl karar verir. Her kategori için dört sinyal harmanlanır: harcama hacmi, harcama trendi (son 45 gün / önceki 45 gün), önümüzdeki dönemin sezonsallığı (SEASONAL_PATTERN öncül tablosu) ve o kategoriyi zaten hedefleyen bir kampanya olup olmadığı. Ağırlıklar appsettings.json üzerinden ayarlanır — şu an "modeli eğitmek" bu ağırlıkları oynatmak demektir. Skorum mantığı tek bir kod bloğunda izole; ileride eğitilmiş bir model bu bloğun yerine geçebilir, uç nokta ve ekran değişmez.

Doğrulama. 14 aylık, 22 kategorili, 49.160 işlemlik sentetik ama gerçekçi bir veri seti üretildi (kategori payları, kategoriye özgü tutar dağılımları, aylık sezonsallık, maaş günü / hafta sonu etkileri, müşteri heterojenliği, %1,5 iade, birkaç kasıtlı yapısal kırılım). Aynı veriden 6 farklı istatistiksel yöntemle (regresyon eğimi, Mann-Kendall trend testi, momentum z-skoru, ampirik sezon endeksi, ...) bağımsız bir sıralama çıkarıldı. Sonra uygulama bu veriye karşı çalıştırılıp motorun sırasıyla karşılaştırıldı.

En önemli üç bulgu:

- Genel uyum güçlü.** Motorun sırası ile bağımsız kompozit sıra arasında **Spearman $\rho = 0,87$** ($p < 0,0001$). İlk 4 kategori birebir aynı (Kırtasiye, Eğitim, Giyim, Havayolları).
- Motorun trend ölçümü kaba.** Tek bir "son yarı / önceki yarı" oranı kullanıyor; istatistiksel anlamlılık testi yok. Bu yüzden bazen gürültüyü gerçek trend sanıyor (Mobilya, Yapı & İnşaat yukarı çıktı) ve bazen kademeli bir yükselişi kaçırıyor (Restoran hak ettiğinden aşağıda).
- Hacim terimi büyük düz/düşen kategorileri taşıyor.** Harcama, tek bir en yoğun kategoriye göre normalize edildiği için, Elektronik ve Turizm gibi cirosu büyük ama trendi düşen kategoriler "önerilir" bölgesinde kalıyor.

En önemli üç öneri (kod değiştirmeden, sadece appsettings.json):

Ayar	Şu an	Öneri
SpendWeight (hacim ağırlığı)	1,0	0,65
TrendWeight (trend ağırlığı)	1,5	1,9
MinimumSpend (alt eşik)	1.000	5.000–10.000

Bu ayarlarla motor, bu veri setinde bağımsız istatistiksel sıralamaya belirgin biçimde yaklaşıyor.

Durum. Backend + frontend + testler tamam (65/65 test geçiyor), Docker'da uçtan uca çalışıyor. "Gerçek eğitme" (istatistiksel yöntemleri motora taşımak / ML) Faz 2 olarak planlandı — bkz. Bölüm B, §B.7.

Bölüm A — Motor nasıl çalışır

A.1 Amaç

Operatöre, son dönem kart harcamalarına bakarak **hangi merchant kategorisinde kampanya tanımlamaya değer olduğunu** sıralı bir liste hâlinde sunmak. Her satır bir kategoridir; yanında bir skor, skoru açıklayan gerekçe alanları ve tek tıkla kampanya formunu dolduran bir **taslak** taşır.

Öneriler hiçbir yerde saklanmaz — her istekte `TRANSACTION` tablosu üzerinden yeniden hesaplanır. Kaydedilen tek şey, operatör "Bu öneriyle kampanya oluştur" deyip formu kaydettiğinde `CAMPAIGN` tablosuna yazılan normal kampanyadır.

Neden heuristik (ML değil). Skorlama açık ve deterministik bir formüldür; ağırlıkları `appsettings.json` üzerinden ayarlanır. Skorlama, `GetSuggestionsAsync` içinde tek bir metoda izole edilmiştir; ileride eğitilmiş bir model bu bloğu controller ve ekran hiç değişmeden değiştirebilir.

A.2 Dosya haritası

Dosya	Sorumluluk
Entities/SeasonalPattern.cs	SEASONAL_PATTERN tablosunun POCO'su: kategori × ay × ağırlık
Data/Configurations/SeasonalPatternConfiguration.cs	EF eşlemesi + HasData seed (takvim öncülleri)
Data/Migrations/20260901083535_AddSeasonalPattern.cs	Tabloyu ve seed satırlarını oluşturan migration
Configuration/RecommendationOptions.cs	Ayarlar (Recommendation bölümü): pencere uzunlukları, ağırlıklar
DTOs/Recommendations/CampaignSuggestionDto.cs	Yanıt tipleri: öneri, gerekçe, taslak
DTOs/Recommendations/RecommendationQueryDto.cs	İsteğe bağlı sorgu parametreleri
Services/Recommendations/CampaignRecommendationService.cs	Skorlama mantığının tamamı
Controllers/CampaignRecommendationsController.cs	GET /api/campaign-recommendations (Admin)
Program.cs	DI kaydı + Configure<RecommendationOptions>
CampaignSystem.Tests/CampaignRecommendationServiceTests.cs	6 entegrasyon testi
frontend/ ... /models/recommendation.ts , services/recommendation.service.ts	DTO karşılığı + uç noktayı saran servis
frontend/ ... /campaigns/campaign-suggestions.ts/.html/.css	Ayrı "Kampanya Önerileri" ekranı
frontend/ ... /campaigns/campaign-form.ts/.html/.css	Formdaki satır-içi öneri paneli + öneriden ön-dolum
docs/analysis/	Bölüm B'nin sentetik veri üretici + istatistiksel analizi (yeniden üretilebilir)

A.3 Girdi verisi — dört tablo

TRANSACTION — ana kaynak. Amount alış satırlarında pozitif, **iade satırlarında negatif**; bu yüzden net harcama = SUM(Amount) (ayrı çıkarma gerekmez). OriginalTransactionId dolu ise satır bir iadedir. MerchantId boş satırlar elenir.

MERCHANT / MERCHANT_CATEGORY — her işlemi Merchant.MerchantCategoryId ile kategoriye bağlar. Kategoriler sabit id'lerle seed'li (1 Gıda/Market ... 22 Eğlence).

SEASONAL_PATTERN (bu özellikle geldi) — takvim önceli: (kategori, ay, ağırlık) . 1,00 sıradan ay; > 1 bilinen sezonsal tepe; < 1 durgunluk. Satırı olmayan çift 1,00 sayılır (~106 satır seed'li). Değerler ölçülmüş değil, **Türkiye perakende sezonsallığına dayalı öncüllerdir**: okula dönüş Ağu–Eyl (Kırtasiye, Eğitim, Elektronik), yaz akaryakıt/seyahat, Kasım elektronik, sezon geçişlerinde giyim, ilkbahar düğün. Bölüm B, §B.5.4'te bu öncüllerin bir sınırı gösteriliyor.

CAMPAIGN / CAMPAIGN_MERCHANT — "kapsam boşluğu" sinyali için: bir kategoride **açık veya yaklaşan** (`IsActive = 1` , `Status <> 'Ended'`) bir kampanya, o kategorideki bir merchant'ı hedefliyor mu? Merchant kriteri hiç olmayan (yatay) kampanya kapsam sayılmaz.

A.4 Ayarlar — RecommendationOptions

`appsettings.json` → `Recommendation` bölümünden bağlanır; tek istek bazında `RecommendationQueryDto` ile geçici ezilebilir.

Ayar	Varsayılan	Anlamı
<code>LookbackDays</code>	90	Harcama ve trendin okunduğu geçmiş penceresi. Ortadan ikiye bölünüp trend okunur.
<code>HorizonDays</code>	45	Önerilen kampanyanın varsayılan süresi. Hangi ayların sezon ağırlığının ortalanacağını belirler.
<code>MinimumSpend</code>	1000	Bu tutarın altında net harcaması olan kategori elenir.
<code>MaxSuggestions</code>	10	Uç noktanın döndüğü en fazla öneri sayısı.
<code>SpendWeight</code>	1.0	Normalize edilmiş harcama hacminin skordaki ağırlığı.
<code>TrendWeight</code>	1.5	Trendin skordaki ağırlığı.
<code>SeasonWeight</code>	1.25	Sezonsal artışın skordaki ağırlığı.
<code>CoverageGapBoost</code>	1.75	Hiçbir açık/yaklaşan kampanyanın kapsamadığı kategoriye uygulanan çarpan.
<code>SuggestedRewardRate</code>	0.02	Kategorinin ortalama işlem tutarının, forma önerilen ödül puanı olarak yansıyan oranı.

A.5 Algoritma — koddan adım adım

Dosya: `Services/Recommendations/CampaignRecommendationService.cs` , metot: `GetSuggestionsAsync` .
Satır numaraları 265 satırlık dosyaya göredir.

Adım 1 — Ayarları çöz, mantıksız değerleri sınırla (satır 34–37)

Her parametre için: istek verilmişse o, yoksa `appsettings` değeri. `Math.Clamp` ile uçuk değerler güvenli aralığa çekilir; dışarıdan gelen sorgu string'i uç noktayı bozamaz.

```
var lookbackDays = Math.Clamp(query.LookbackDays ?? _options.LookbackDays, 14, 365);
var horizonDays = Math.Clamp(query.HorizonDays ?? _options.HorizonDays, 7, 180);
var minimumSpend = Math.Max(0m, query.MinimumSpend ?? _options.MinimumSpend);
var maxSuggestions = Math.Clamp(query.MaxSuggestions ?? _options.MaxSuggestions, 1, 50);
```

Adım 2 — Zaman pencerelerini kur (satır 39–42)

`midPoint` pencereyi ikiye böler: `midPoint ... now` **son yarı**, `windowStart ... midPoint` **önceki yarı**. Trend bu ikisinin karşılaştırmasıdır. `now ... horizonEnd` önerilen kampanyanın varsayımsal süresidir; sezon ağırlığı bu aralığın aylarından hesaplanır.

```
var now          = DateTime.Now;
var windowStart  = now.AddDays(-lookbackDays);
var midPoint     = now.AddDays(-lookbackDays / 2.0);
var horizonEnd   = now.AddDays(horizonDays);
```

Adım 3 — İşlemleri kategori bazında topla (satır 47–61)

Tek sorgu, SQL Server'a **tek bir** GROUP BY olarak çevrilir. Kategori başına dört sayı:

```
var aggregates = await context.Transactions
    .AsNoTracking()
    .Where(t => t.MerchantId != null
        && t.TransactionDate >= windowStart
        && t.TransactionDate < now)
    .GroupBy(t => t.Merchant!.MerchantCategoryId)
    .Select(g => new CategoryAggregate
    {
        CategoryId    = g.Key,
        RecentSpend   = g.Sum(x => x.TransactionDate >= midPoint ? x.Amount : 0m),
        PriorSpend    = g.Sum(x => x.TransactionDate < midPoint ? x.Amount : 0m),
        PurchaseSpend = g.Sum(x => x.OriginalTransactionId == null ? x.Amount : 0m),
        PurchaseCount = g.Sum(x => x.OriginalTransactionId == null ? 1 : 0)
    })
    .ToListAsync(cancellationToken);
```

- **RecentSpend / PriorSpend** — iki yarının net toplamı. İade satırları negatif olduğu için koşullu SUM onları da netler.
- **PurchaseSpend / PurchaseCount** — yalnızca alış satırları; önerilen ödül puanını boyutlamak için (iadelerden arınmış "gerçek" ciro).
- **NetSpend = RecentSpend + PriorSpend** (satır 262'de türetilir).

Pencerede hiç işlem yoksa boş liste döner.

Adım 4 — Yardımcı sözlükleri yükle (satır 68–99)

Dört ek sorgu, hepsi bellek içi sözlüğe:

Sözlük	İçerik	Amaç
categoryNames	{id: ad}	Başlık ve DTO için
activeMerchantsByCategory	{kategori: [merchantId...]}	Taslağın merchant kriteri
seasonalWeights	{{kategori, ay}: ağırlık}	Sezon araması
coveringCampaigns	{kategori: [campaignId...]}	Kapsam boşluğu tespiti

Adım 5 — Ufuk aylarını bul (MonthsSpanned , satır 194–207)

from 'un ayının 1'inden to 'yu geçene kadar ay ay ilerler. Örnek: bugün 1 Eylül, ufuk 45 gün → [9, 10] .

Adım 6 — Normalizasyon tabanı (satır 101–106)

```
var maxNetSpend = aggregates.Max(a => a.NetSpend);
```

Tüm kategoriler arasındaki en yüksek net harcama. Her kategorinin harcaması buna bölünerek 0–1'e normalize edilir. (Bu adımın bir sınırı Bölüm B, §B.5.3'te.)

Adım 7 — Her kategori için skor (satır 110–179)

7a. Eşik filtresi. `NetSpend < MinimumSpend` ise kategori atlanır.

7b. Kapsam boşluğu. Kategoriyi hedefleyen açık/yaklaşan kampanya varsa ve `includeCovered` verilmemişse kategori listeden çıkarılır.

7c. Trend oranı (satır 125–127).

```
var trendRatio = aggregate.PriorSpend > 0m
    ? (double)((aggregate.RecentSpend - aggregate.PriorSpend) / aggregate.PriorSpend)
    : (double?)null;
```

(son yarı - önceki yarı) / önceki yarı. 0,42 → %42 artış. Önceki yarı sıfır/negatifse null (skorlamada 0 gibi davranır).

7d. Sezon ağırlığı (satır 129–132). Ufuk aylarının her biri için `SEASONAL_PATTERN` ağırlığı (yoksa 1,0), sonra ortalama. Örnek — Kırtasiye, aylar [9, 10]: Eylül 1,60, Ekim 0,90 → **1,25**.

7e. Skor formülü (satır 137–143).

```
var normalisedSpend = (double)(aggregate.NetSpend / maxNetSpend); // 0 ... 1
var clampedTrend     = Math.Clamp(trendRatio ?? 0.0, -1.0, 3.0);    // -%100 ... +%300

var rawScore =
    _options.SpendWeight * normalisedSpend
  + _options.TrendWeight * clampedTrend
  + _options.SeasonWeight * (seasonalWeight - 1.0);

var score = Math.Max(rawScore, 0.01)
    * (isCoverageGap ? _options.CoverageGapBoost : 1.0);
```

Terim	Açılım	Anlamı
Harcama	$1,0 \times \text{normalisedSpend}$	En yoğun kategori 1,0 puan; yarısı kadar harcayan 0,5
Trend	$1,5 \times \text{clamp}(\text{trend}, -1, 3)$	+%100 büyüyen +1,5; %50 küçülen -0,75
Sezon	$1,25 \times (\text{sezonAğırlığı} - 1)$	Ağırlık 1,4 → +0,5; 0,8 → -0,25; 1,0 → 0

`Math.Max(raw, 0,01)` : üç terim de negatife giderse skoru küçük pozitifte tutar (zayıf kategoriler dipte toplanır). `CoverageGapBoost` (1,75): kapsanmayan kategori, aynı ham skorlu kapsanan kategorinin önüne geçer.

7f. Önerilen ödül puanı (satır 145–149). Kategorinin ortalama alış tutarı $\times$ `SuggestedRewardRate`, tam sayıya yuvarlanır, en az 1. Bağlayıcı değil.

7g. DTO'nun kurulması (satır 151–178): skor, insan-okunur başlık cümlesi (`BuildHeadline`), ham gerekçe alanları ve form ön-dolum taslağı.

Adım 8 — Sırala, kes, numaralandır (satır 181–189)

```
var ranked = scored.OrderByDescending(s => s.Score).Take(maxSuggestions).ToList();
for (var i = 0; i < ranked.Count; i++) ranked[i].Rank = i + 1;
```

Başlık cümlesi (`BuildHeadline` , satır 209–243)

Eşikler (%15 trend, 1,1/0,9 sezon) cümleye yalnız anlamlı sinyalleri koyar; küçük dalgalanmalar metne girmez. Örnek çıktı:

"Kırtasiye kategorisinde son 90 günde 163.058 ₺ harcama, harcama %137 arttı, önümüzdeki dönem sezonsal olarak yüksek — bu kategoride aktif kampanya yok."

A.6 Sayısal örnek

Bölüm B'nin veri setiyle (bugün 1 Eylül, varsayılan ayarlar). Bu veri setinde en yüksek net harcama **Turizm**'de (1.542.283 ₺) → `maxNetSpend` bu.

Kırtasiye — skor 4,32, sıra #1

Bileşen	Hesap	Değer
NetSpend		163.058 ₺
normalisedSpend	$163.058 / 1.542.283$	0,106
trend	son yarı $\approx$ 2,4× önceki yarı	+1,366
seasonalWeight	(Eyl 1,60 + Eki 0,90) / 2	1,25
Harcama terimi	$1,0 \times 0,106$	0,106
Trend terimi	$1,5 \times 1,366$	2,049
Sezon terimi	$1,25 \times (1,25 - 1,0)$	0,313
rawScore	toplam	2,468
score	$2,468 \times 1,75$ (kapsam boşluğu)	4,32

Turizm — skor 1,23, sıra #9 (ama trendi düşüşte)

Bileşen	Hesap	Değer
NetSpend (en yüksek)		1.542.283 ₺
normalisedSpend	$1.542.283 / 1.542.283$	1,000
trend	son yarı < önceki yarı	-0,261
seasonalWeight	$(\text{Eyl } 1,15 + \text{Eki } 1,00) / 2$	1,075
Harcama terimi	$1,0 \times 1,0$	1,000
Trend terimi	$1,5 \times (-0,261)$	-0,391
Sezon terimi	$1,25 \times (1,075 - 1,0)$	0,094
rawScore	toplam	0,703
score	$0,703 \times 1,75$	1,23

Ne anlama geliyor. Turizm bu veri setinin en yüksek ciro lu kategorisi olduğu için harcama terimi tek başına 1,0 puan getiriyor ve skoru "önerilir" bölgesinde tutuyor — oysa Turizm son 45 günde belirgin düşüşte. Bu, Bölüm B'de incelenen ana ayrışmalardan biri.

A.7 Önemli tasarım kararları

Durum / karar	Davranış	Gerekçe
Pencerede hiç işlem yok	<code>[]</code>	Ekranda "Şu an öne çıkan bir kategori yok."
<code>PriorSpend ≤ 0</code>	<code>trendRatio = null →</code> skorlamada 0	Sıfıra bölme yok; "sonsuz büyüme" abartısı yok
Üç terim de negatif	<code>score = max(raw, 0, 01) ×</code> <code>boost</code>	Zayıflar dipte toplanır, sıralama bozulmaz
Yatay (merchant kriteri olmayan) kampanya	Kapsam sayılmaz	Amaç <i>kategoriye hedefli</i> kampanya önermek
Repository yerine doğrudan <code>DbContext</code>	Çok tablolu <code>GROUP BY</code>	Gruplama gerektiren okuma repository işi değil
Skorlama tek metotta izole	—	Faz 2'de model bu bloğun yerine geçer

Bölüm B — Doğrulama: sentetik veri + istatistiksel analiz

B.1 Neden ve nasıl test ettik

Motor bir formül; formülün "doğru" kararlar verip vermediğini görmek için, cevabı önceden bilinen bir veriye ihtiyaç var. Bu yüzden:

1. **Sıfırdan gerçekçi bir harcama veri seti ürettik** — hangi kategorinin artışta, hangisinin düşüşte, hangisinin sadece cirosu büyük olduğunu biz belirledik (ve bazı tuzaklar koyduk).
2. **Aynı veriden, motora hiç bakmadan, 6 farklı istatistiksel yöntemle** bağımsız bir "hangi kategoride kampanya açılmalı" sıralaması çıkardık.
3. **Uygulamayı bu veriye karşı çalıştırıp** motorun sırasıyla bizim sıramızı karşılaştırdık.

Tüm bunlar `docs/analysis/generate_and_analyze.py` ve `compare.py` içinde, seed `20260901` ile **deterministik** ve yeniden üretilebilir (§B.8).

B.2 Sentetik veri seti — "mümkün olduğunca gerçek"

Boyut	Değer
İşlem	49.160 (48.434 alış + 726 iade)
Toplam net harcama	54.417.841 ₺
Zaman aralığı	2025-07-03 ... 2026-09-01 (425 gün ≈ 14 ay)
Müşteri / Kart / Merchant	451 / ~700 / 88 (22 kategori × 4)
Kapsayan kampanya	3 (Gıda/Market, Akaryakıt — açık; Sağlık — yaklaşan)

Gerçekçiliği sağlayan yedi yapı:

1. **Kategori işlem payı** — Türkiye kart kullanımına yakın: Gıda/Market %20,5, Restoran %15, Akaryakıt %9, Sağlık/Eczane %6, Telekom %4,5 ... Elektronik %2, Beyaz Eşya %0,7.
2. **Kategoriye özgü tutar dağılımı** — her kategori **lognormal** (medyan + şekil). Gıda medyan 220 ₺; Beyaz Eşya 9.500 ₺; Elektronik 3.800 ₺ ve ağır sağ kuyruk (büyük alışveriş nadir).
3. **Aylık sezonsallık** — her kategoriye 12 aylık çarpan: yazın Akaryakıt/Turizm/ Havayolları tepe; Kasım Elektronik; ilkbahar+sonbahar Giyim; Ağu–Eyl Eğitim/Kırtasiye; Ocak Spor; ilkbahar Kuyum.
4. **Ay içi yoğunluk** — maaş/ay sonu günleri ×1,35; hafta sonu ×1,15. İşlem sayısı Poisson.
5. **Müşteri heterojenliği** — Pareto benzeri harcama ağırlıkları: az sayıda ağır harcayan, çoğunluk hafif.
6. **İadeler** — işlemlerin %1,5'i, yüksek biletli kategorilere 3× eğilimli, negatif tutarla ve `OriginalTransactionId` ile orijinale bağlı.
7. **Kasıtlı yapısal kırılımlar** (sezondan bağımsız, gerçek trendler — motorun bunları yakalayıp yakalamadığını test etmek için):

Kategori	Kırılım	Ne test ediyor
Kozmetik	son 55 günde $\times 1,9$ artış, SEASONAL_PATTERN'de satırı yok	Motor sezon yardımı olmadan gerçek trendi yakalıyor mu?
Turizm	son 45 günde $\times 0,55$ düşüş, ama Eylül sezon önceli +1,15	Motorun pozitif sezon önceli, cari yıldaki düşüşü örtüyor mu?
Eğitim	son 40 günde $\times 1,6$	Sezon + gerçek trend birlikte $\rightarrow$ tepe beklenir
Kırtasiye	son 38 günde $\times 1,7$	Aynı

B.3 Bağımsız istatistiksel yöntemler

Hepsi son 90 günlük pencerede (motorun `LookbackDays` 'i), kategori bazında:

#	Yöntem	Ne ölçer	Motordan farkı
M1	Net harcama	<code>SUM(Amount)</code>	Motorun harcama terimiyle aynı
M2	İki-yarı oranı	$(\text{son}45g - \text{önceki}45g) / \text{önceki}45g$	Motorun kullandığı yöntem
M3	OLS eğim	Günlük harcamaya en küçük kareler doğrusu: eğim + p-değeri + R^2	Motor eğim/anlamlılık kullanmıyor
M4	Mann-Kendall	Haftalık harcamaya parametrik olmayan trend testi (Kendall τ + p)	Motorda yok
M5	Momentum z	$(\text{son } 30g \text{ ort.} - \text{önceki } 60g \text{ ort.}) / \text{önceki } 60g \text{ std}$	Motorda yok
M6	Ampirik sezon	14 aylık geçmişten ölçülmüş ay-endeksi + 45 günlük projeksiyon	Motor öncül tablo kullanıyor, ölçüm değil
M7	Kapsam	Açık/yaklaşan kampanyanın hedeflediği kategori	Motorla aynı

Kompozit bağımsız skor: $\{\ln(\text{net harcama}), \text{OLS eğim}, \text{MK } \tau, \text{momentum } z, \text{ampirik sezon}\}$ z-normalize edilir; trend bloğu üç yöntemin **ortalaması** (motorun tek oranına karşı); ağırlıklar $0,9 \cdot \text{harcama} + 1,7 \cdot \text{trend} + 1,1 \cdot \text{sezon}$; kapsam boşluğu $\rightarrow \times 1,75$. Yani motorla **aynı ruhta** ama trendi tek orana değil üç kanıta, sezonu öncül tabloya değil ölçülen endekse dayandırır.

B.4 Sonuç — sıralama uyumu

Karşılaştırma **kapsam boşluğu olan 19 kategori** üzerinden (kapsanan 3'ü ikisi de eler).

Spearman $\rho = 0,872$ ($p < 0,0001$) — motorun sırası ile bağımsız kompozit sıra arasında güçlü uyum. **İlk 4 birebir aynı.**

Motor skoru ile tek tek yöntemler arasındaki yön uyumu:

Bağımsız yöntem	Motor skoruyla Spearman ρ
M1 net harcama	+0,74
M2 iki-yarı oranı	+0,78
M3 OLS eğim	+0,55
M4 Mann-Kendall τ	+0,49
M5 momentum z	+0,52
M6 ampirik sezon	+0,31

Ne anlama geliyor. Motorla en yüksek uyum M1/M2 ile — beklenen, çünkü motorun skoru bu ikisinin ağırlıklı toplamı. Regresyon / Mann-Kendall / momentum ile uyum daha düşük: motor trendi tek bir orana indirger, istatistiksel anlamlılık kullanmaz.

İlk 12 kategori — yan yana

(p) = istatistiksel anlamlılık; 0,05'in altı "gerçek trend", üstü "ayırt edilemez".

# motor	# bağımsız	Δ	Kategori	Motor skor	Net harcama	Motor trend	OLS eğim (p)	MK τ (p)	Mom. z	Ampirik sezon	Motor sezon önceli
1	1	0	Kırtasiye / Oyuncak	4,32	163.058	+1,37	+1,89 (0,00)	+0,67 (0,00)	+2,45	1,11	1,25
2	2	0	Eğitim	2,66	874.471	+0,35	+0,53 (0,07)	+0,24 (0,31)	+0,84	1,26	1,35
3	3	0	Giyim	1,85	562.924	+0,30	+0,41 (0,01)	+0,27 (0,25)	+0,46	1,19	1,20
4	4	0	Havayolları / Ulaşım	1,75	566.717	+0,42	+0,62 (0,07)	+0,48 (0,03)	+0,43	1,04	1,00
5	8	-3	Elektronik	1,43	1.149.373	-0,04	-0,02 (0,97)	-0,09 (0,74)	+0,22	0,98	1,10
6	13	-7	Mobilya & Ev Tekstili	1,39	621.311	+0,22	+0,18 (0,68)	+0,09 (0,74)	+0,04	0,92	1,05
7	5	+2	Ayakkabı & Aksesuar	1,30	166.843	+0,28	+0,27 (0,40)	+0,18 (0,46)	-0,02	1,18	1,18
8	12	-4	Yapı & İnşaat	1,24	498.080	+0,20	+0,14 (0,68)	+0,09 (0,74)	-0,04	0,97	1,08
9	9	0	Turizm / Seyahat / Otel	1,23	1.542.283	-0,26	-0,28 (0,38)	-0,36 (0,12)	-0,26	1,05	1,08
10	6	+4	Kozmetik	1,22	183.098	+0,38	+0,51 (0,03)	+0,24 (0,31)	+0,51	1,07	1,00
11	7	+4	Restoran / Yeme-İçme	0,98	687.893	+0,08	+0,24 (0,03)	+0,33 (0,15)	+0,40	0,93	1,00
12	11	+1	Eğlence	0,81	138.660	+0,25	+0,31 (0,13)	+0,36 (0,12)	+0,36	1,01	1,00

B.5 Belirgin ayrışmalar — her biri dört satırda

B.5.1 Mobilya #6→#13, Yapı & İnşaat #8→#12 — sahte trend

- **Motor ne dedi:** İki-yarı oranı +0,22 / +0,20 → "artışta", yukarı çekti.
- **Bağımsız ne dedi:** OLS eğim $p = 0,68$ (anlamsız), MK $\tau \approx +0,09$, momentum $\approx 0 \rightarrow$ "gerçek trend yok".
- **Neden:** İki-yarı oranı, iki toplam arasındaki **gürültülü bir farktır ve anlamlılık testi yoktur**. 45'er günlük iki toplamda rastgele dalgalanma %20'lik "artış" gibi görünür.
- **Anlamı:** Motor, gürültüyü gerçek momentum sanıp bu kategorileri hak etmedikleri sıraya koyuyor.

B.5.2 Restoran #11→#7 — kademeli trendi kaçırma

- **Motor ne dedi:** İki-yarı oranı sadece +0,08 (zayıf).
- **Bağımsız ne dedi:** OLS eğim +0,24, **$p = 0,03$ (anlamlı)**; MK $\tau +0,33$; momentum +0,40 → gerçek, istikrarlı yükseliş.

- **Neden:** İki-yarı oranı **doğrusal/kademeli artışa duyarsızdır**. Harcama tüm pencere boyunca yavaşça artıyorsa iki yarının toplamı birbirine yakın çıkar; günlük seriye çizilen doğrunun eğimi bunu net yakalar.
- **Anlamı:** Motor, yavaş ama sağlam büyüyen bir kategoriye listede aşağıda tutuyor.

B.5.3 Elektronik #5→#8 ve Turizm (aynı sıra, zıt karar) — hacim terimi büyüklüğü taşıyor

- **Motor ne dedi:** Elektronik skor 1,43 (ciro 1,15 M); Turizm skor **+1,23** → ikisi de "önerilir" bölgesinde.
- **Bağımsız ne dedi:** Elektronik her yöntemde ~0/negatif trend → #8; Turizm son 45 günde düşüşte (OLS -0,28, MK τ -0,36, momentum -0,26), kompozit skoru **-0,46** → önerilmez.
- **Neden:** `SpendWeight` terimi harcamayı **tek bir en yoğun kategoriye** göre normalize eder. Turizm bu veri setinin en büyüğü → hem tavanı belirliyor hem kendi payı 1,0 oluyor → harcama terimi tek başına skoru eşğin üstünde tutuyor.
- **Anlamı:** Ciro su büyük ama düşen/düz bir kategori, sırf büyüklüğü yüzünden öneri listesine giriyor.

B.5.4 Turizm — sezon önceli cari yıl kırılımını göremiyor

- **Motor ne dedi:** `SEASONAL_PATTERN` Turizm/Eylül = 1,15 → sezon terimi **+0,09** (skoru yukarı itti).
- **Bağımsız ne dedi:** Bu yılın verisinde Turizm Eylül'de düşüşte; ampirik sezon endeksi yalnız 1,05, gerçek trend sert negatif.
- **Neden:** Öncül tablo statiktir. "Eylül bir Turizm ayıdır" iyi bir uzun dönem ortalaması olabilir ama **bu yıl** yaz erken bittiyse öncül yanıltır.
- **Anlamı:** Sezon terimi bazen yanlış yöne itiyor; ampirik endeks + gerçek trend sinyali bunu düzeltir.

B.5.5 Kozmetik #10→#6 — motor küçük ama hızlı büyüyen az ödüllendiriyor

- **Motor ne dedi:** Trendi yakaladı (+0,38) ama #10'a koydu — ciro su küçük (183 K → `normalisedSpend` ≈ 0,12) ve sezon önceli yok (tam 0 katkı).
- **Bağımsız ne dedi:** OLS eğim +0,51 (**p = 0,03**), momentum +0,51 → güçlü büyüme; kompozit #6.
- **Neden:** Motorun `SpendWeight` 'i mutlak ciroyu fazla ödüllendirip momentum'u bastırıyor; bağımsız kompozit $\ln(\text{harcama})$ ile hacim avantajını sıkıştırıyor.
- **Anlamı:** Hızlı büyüyen bir niş bir kampanyayı hak edebilir; motor bunu geç fark ediyor.

B.6 Ayar önerileri — kod değiştirmeden

`appsettings.json` → `Recommendation`:

Ayar	Şu an	Öneri	Gerekçe
<code>SpendWeight</code>	1,0	0,6–0,7	§B.5.3: hacim terimi büyük düz/düşen kategorileri taşıyor
<code>TrendWeight</code>	1,5	1,8–2,0	Trend, kampanya zamanlaması için hacimden daha bilgilendirici
<code>SeasonWeight</code>	1,25	1,1–1,25	Öncül statik ve bazen yanıltıcı (§B.5.4) — ağırlığını çok artırma
<code>MinimumSpend</code>	1000	5.000–10.000	Çok küçük kategorileri baştan eler, gürültüyü azaltır

`SpendWeight` 1,0 → 0,65 ve `TrendWeight` 1,5 → 1,9 yapmak bu veri setinde Elektronik'i #5 → ~#8'e, Turizm'i #9 → ~#12'ye çeker — bağımsız sıraya yaklaşıp.

B.7 Faz 2 — "gerçek eğitime"

Değiştirilecek tek yer `GetSuggestionsAsync` içindeki `rawScore` bloğu (A.5, Adım 7e). Bu analizden çıkan üç net iyileştirme:

1. **Trendi tek orandan çıkar.** İki-yarı oranı hem sahte pozitif üretiyor (§B.5.1) hem kademeli trendi kaçıyor (§B.5.2). Yerine günlük seriye **OLS eğim + p-değeri** (anlamsızsa katkı 0) veya Mann-Kendall τ . Servis, kategori $\times$ gün toplamalarını (birkaç bin satır) bellek içine çekip basit bir OLS ile hesaplayabilir.
2. **Hacim terimini logaritmik/robust normalize et.** $\ln(1+net)$ / $\ln(1+max)$ veya yüzdelik sıra. Tek dev kategori artık tavanı domine etmez (§B.5.3).
3. **Sezonu veriden öğren.** `SEASONAL_PATTERN` 'i elle seed yerine, yeterli geçmiş biriktiğinde her (kategori, ay) için `ay_ortalama_günlük_harcama` / `yıllık_ortalama` hesaplayıp tabloya yaz (kısmi ayları atarak). Öncül yalnız veri yetersizken kullanılır.

Bu üç değişiklik, §B.5'teki 6 ayrışmanın 5'ini bağımsız yöntemlerle hizalar. Arayüz sözleşmesi (`ICampaignRecommendationService` , `CampaignSuggestionDto`) ve HTTP/ekran katmanı değişmez.

B.8 Yeniden üretme

```
# 1) Veri seti + bağımsız analiz → docs/analysis/_out/*.json, dataset.sql
python docs/analysis/generate_and_analyze.py

# 2) Taze DB: şema + seed + veri
$env:ConnectionString__DefaultConnection = "Server=localhost\SQLEXPRESS;Database=CampaignSystem_AnalysisTest"
$env:Jwt__SigningKey = "0123456789012345678901234567890123456789"
dotnet ef database update --project CampaignSystem --startup-project CampaignSystem
# dataset.sql'de USE CampaignSystem → USE CampaignSystem_AnalysisTest yapıp sqlcmd ile yükle

# 3) Uygulamayı o DB'ye karşı çalıştır, GET /api/campaign-recommendations
# → docs/analysis/_out/app_ranking_all.json

# 4) Karşılaştır → docs/analysis/karsilastirma-raporu.md
python docs/analysis/compare.py
```

Ekler

Ek 1 — HTTP katmanı

`Controllers/CampaignRecommendationsController.cs`

```
[ApiController]
[Authorize(Roles = "Admin")]
[Route("api/campaign-recommendations")]
public class CampaignRecommendationsController(ICampaignRecommendationService recommendationService)
    : ControllerBase
{
    [HttpGet]
    [ProducesResponseType<List<CampaignSuggestionDto>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<List<CampaignSuggestionDto>>> GetAll(
        [FromQuery] RecommendationQueryDto query, CancellationToken cancellationToken)
        => Ok(await recommendationService.GetSuggestionsAsync(query, cancellationToken));
}
```

Yalnız Admin rolü (token yoksa 401). Opsiyonel sorgu parametreleri: `lookbackDays`, `horizonDays`, `minimumSpend`, `maxSuggestions`, `includeCovered`. Öneri yoksa `[]`.

Ek 2 — Frontend akışı

- `recommendation.service.ts` — `getSuggestions(query)` → uç noktayı çağırır.
- Ayrı ekran** (`campaign-suggestions.*`, route `campaigns/suggestions`, sol menüde "Kampanya Önerileri") — öneri kartları + "Bu öneriyle kampanya oluştur".
- Satır-içi panel** (`campaign-form.*`) — kampanya oluşturma ekranı **her açıldığında** `getSuggestions({ maxSuggestions: 6 })` çağrılır, formun üstünde açılır-kapanır bir panel gösterilir; her satırda "Uygula" formu yerinde doldurur. Motor canlı hesapladığı için bu her zaman güncel.
- Öneriden forma geçiş** — ayrı ekrandaki buton `router.navigate` ile taslağı `history.state`'te taşır; `campaign-form.ts` → `applyDraft()` alanları `patchValue` ile doldurur, `selectedMerchants` signal'ına merchant id'leri yazar.

Ek 3 — Testler

`CampaignSystem.Tests/CampaignRecommendationServiceTests.cs` — gerçek SQL Server'a karşı, her test kendi rollback'li transaction'ında.

Test	Kanıtlandığı
<code>Suggests_ABusyCategory_NoCampaignCovers</code>	Yoğun + kampanyasız kategori #1, <code>IsCoverageGap=true</code> , taslak dolu
<code>Excludes_ACategory_AnOpenCampaignAlreadyCovers</code>	Kapsanan kategori varsayılan listede yok; <code>includeCovered=true</code> ile geri gelir
<code>NetsRefundsOut_OfTheSpendItScores</code>	10.000 alış – 6.000 iade → <code>TotalSpend = 4.000</code>
<code>ReportsARisingTrend_InTheReasonAndHeadline</code>	Son yarıya yığılan harcama → <code>TrendRatio > 0.5</code> , başlıkta "arttı"
<code>RanksASeasonalCategory_AboveANeutralOne_AtEqualSpend</code>	Sezonlu kategori, eşit harcamalı nötrden yüksek
<code>ReturnsEmpty_WhenNoSpendFallsInsideTheWindow</code>	Pencere dışı işlem → <code>[]</code>

Ek 4 — Motorun eşdeğer tek SQL sorgusu (bilgi amaçlı)

```
DECLARE @now datetime2 = SYSDATETIME();
DECLARE @start datetime2 = DATEADD(DAY, -90, @now);
DECLARE @mid datetime2 = DATEADD(DAY, -45, @now);

;WITH agg AS (
    SELECT m.MerchantCategoryId AS CategoryId,
           SUM(CASE WHEN t.TransactionDate ≥ @mid THEN t.Amount ELSE 0 END) AS RecentSpend,
           SUM(CASE WHEN t.TransactionDate < @mid THEN t.Amount ELSE 0 END) AS PriorSpend
    FROM [TRANSACTION] t JOIN MERCHANT m ON m.Id = t.MerchantId
    WHERE t.TransactionDate ≥ @start AND t.TransactionDate < @now
    GROUP BY m.MerchantCategoryId
),
season AS (
    SELECT MerchantCategoryId, AVG(Weight) AS SeasonalWeight
    FROM SEASONAL_PATTERN WHERE Month IN (9, 10) GROUP BY MerchantCategoryId
),
covered AS (
    SELECT DISTINCT m.MerchantCategoryId
    FROM CAMPAIGN_MERCHANT cm
    JOIN CAMPAIGN c ON c.Id = cm.CampaignId
    JOIN MERCHANT m ON m.Id = cm.MerchantId
    WHERE c.IsActive = 1 AND c.Status <> 'Ended'
)
SELECT a.CategoryId, mc.CategoryName,
       (a.RecentSpend + a.PriorSpend) AS NetSpend,
       CASE WHEN a.PriorSpend > 0
            THEN (a.RecentSpend - a.PriorSpend) / a.PriorSpend END AS TrendRatio,
       COALESCE(s.SeasonalWeight, 1.0) AS SeasonalWeight,
       CASE WHEN cv.MerchantCategoryId IS NULL THEN 1 ELSE 0 END AS IsCoverageGap
FROM agg a
JOIN MERCHANT_CATEGORY mc ON mc.Id = a.CategoryId
LEFT JOIN season s ON s.MerchantCategoryId = a.CategoryId
LEFT JOIN covered cv ON cv.MerchantCategoryId = a.CategoryId
WHERE (a.RecentSpend + a.PriorSpend) ≥ 1000
ORDER BY NetSpend DESC;
```

Skor karışımı (normalize, clamp, ağırlıklar, boost) ve başlık cümlesi kasıtlı olarak uygulama tarafında tutulur — ayarlanabilir kalması ve ileride bir modelle değiştirilebilmesi için.

Hazırlanma: 2026-09-01 · Dal `feature/campaign-recommendations` · Kod referansları

`CampaignRecommendationService.cs` (265 satır) satır numaralarına göre · Doğrulama seed'i 20260901, Spearman $\rho(\text{motor}, \text{bağımsız kompozit}) = 0,872$.